Thesis: Toward-verification-of-QUIC-extensions
Report I

Academic year : 2020 - 2021

Teacher :
Bonaventure Olivier, Legay Axel
Piraux Maxime, Rochet Florentin
Course : Thesis
Collaborators :
Crochet Christophe, Sambon Jean-François

Contents

1 Introduction **1**

2 Results **1**
2.1 Methodology 1
2.2 Our results 2
2.2.1 Summary of errors 3
2.2.2 Explanation of errors 4
2.3 Comparison between our results and McMillan results 7

3 Conclusion **8**
3.1 Next in the work 8

1 Introduction

In this report we are going to show you the results we obtained by following the experimental protocol of this paper:

"Formal Specification and Testing of QUIC" from *KL McMillan, LD Zuck*[1].

We will first present you our results for the draft 18 of QUIC such as presented in the paper and in the McMillan repository. Then, we will compare the results we obtained with the results that were obtained in this paper. This part of the project was intended to show that Ivy was an effective way, even if experimental, to formally test the QUIC protocol and potentially its extensions.

2 Results

2.1 Methodology

We begin this task by creating "installer" files[2] in order to try to have similar setup than McMillan by checkout out good commit for each modules/third party.

But we don't have any guarantee that the setup is exactly the same than McMillan which could change the final result. We also need to say that installing everything was very annoying since a lot of bugs depending on the version were occurring so we have to find the best commit possible. Indeed, McMillan didn't precise explicitly the commits he were using in order to get his results for the QUIC version as well as the `Ivy` version.

Moreover, since McMillan tested "*evolving QUIC version and specification*", so also multiple drafts of QUIC were tested so either older or equal than the draft 18, which could again explain why we might get different result. We can explain the fact that we don't test all version by the fact that it's very difficult to get the good commit of each modules to get the same result so we want to get first result as soon as possible and then extend it for the current QUIC draft.

We first installed `Ivy` at the following branch `quic18_client` which mainly implement the QUIC draft 14 with some updated rules (so old specification might not work but this normal, but we discover all of that too late). There are also other QUIC 18 branches but those were bugging a lot. We also decide to change a little bit the normal installation setup since it was not adapted for the draft 18 due to version errors, all of that can be checked in the installers.

Then we installed different version of QUIC but only tested the 3 first one:

1. `PicoQUIC` at the following commit `95dd82f`[3]
    - `PicoTLS 4e6080b6a1ede0d3b23c72a8be73b46ecaf1a084`
2. `Quant b55051011a3a040ccc93e83add29dec46eceda54` [4]
3. `MinQUIC` (not updated since 2018)[5]
4. `Google QUIC`, note that for installing chromium on Linux Mint we modified the building module file since Mint is not an official distribution.

[1] `http://mcmil.net/pubs/SIGCOMM19.pdf`
[2] `https://github.com/ElNiak/Toward-verification-of-QUIC-extensions/tree/master/installer`
[3] `https://github.com/private-octopus/picoquic`
[4] `https://github.com/NTAP/quant`
[5] `https://github.com/ekr/minq`

As McMillan did in the paper, we ran a series of 1000 tests for each test to experiment on the long run the protocol implementation. We also do that on 3 machines:

1. One virtual machine on Ubuntu 18.0 LTS with virtual box
2. One recent Dell with Linux Mint 19.3 Tricia
3. And finally one old Asus with Linux Mint 19 Tara

2.2 Our results

Our results are in the following repository:

`https://github.com/ElNiak/Toward-verification-of-QUIC-extensions/tree/master/result`

We did our tests on different setup. Our results are located in the result folder. Inside it, there is one folder containing the results of each 1000 iterations per test for all machine configuration.

Table 1: **VM Ubuntu**

	PicoQUIC	Quant	MinQUIC
quic_server_test_stream	386	638	1000
quic_server_test_max	318	694	1000
quic_server_test_connection_close	908	922	1000
quic_client_test_max	867	1000	1000

Table 2: **Asus Linux Mint**

	PicoQUIC	Quant	MinQUIC
quic_server_test_stream	390	614	1000
quic_server_test_max	380	703	1000
quic_server_test_connection_close	899	915	1000
quic_client_test_max	862	1000	1000

Table 3: **Dell Linux Mint**

	PicoQUIC	Quant	MinQUIC
quic_server_test_stream	353	649	1000
quic_server_test_max	416	727	1000
quic_server_test_connection_close	907	914	1000
quic_client_test_max	870	1000	1000

Here we can see that independently of the computer distribution, the tests generally produce the same amount of errors per implementation and per test which comfort us in the fact that our result are trustful.

However as we can also see, `MinQUIC` implementation doesn't work but we will see later why.

We can also see that there are a lot of errors in the tested implementations, some of them have more errors than other but this is still impressive. We could easily test other implementation, we just need to find the good commit for the draft 18 but this will be done in future work.

`MinQUIC` case First, as you can see, we were not able to reproduce result for the `MinQUIC` implementation for multiple reason. First this implementation is not maintained anymore and there is no implementation for version 18 of QUIC. The last draft implemented is for QUIC 15, so for that we also checkout the version 15 of `Ivy` but however, we think that the installer of dependency of Ivy were not optimal in the sens that it does not download them with good version we believe since strange code error such wrong number of argument for some function and other were appearing which were corresponding to more recent version of `Ivy`.

2.2.1 Summary of errors

Now let's see in more details the errors we obtain for the different tests on the Virtual Machine, we do not show the results for the other one but you can assume that they are quite similar and can be found here.[6]

Table 4: **VM Ubuntu - `PicoQUIC`**

Test	Error code	#
quic_server_test_stream	quic_connection.ivy: line 753: error: assumption failed	242
	Ran out of values for type cid	121
	quic_connection.ivy: line 1612: error: assumption failed	24
	bind failed: Address already in use	2
	quic_connection.ivy: line 732: error: assumption failed	1
		390/1000
quic_server_test_max	quic_connection.ivy: line 1257: error: assumption failed	219
	Ran out of values for type cid	102
	quic_connection.ivy: line 1612: error: assumption failed	8
	quic_connection.ivy: line 753: error: assumption failed	12
	quic_server_test.ivy: line 629: error: assumption failed	31
	quic_connection.ivy: line 732: error: assumption failed	7
	bind failed: Address already in use	1
		380/1000
quic_server_test_connection_close	Ran out of values for type cid	412
	quic_connection.ivy: line 753: error: assumption failed	53
	quic_connection.ivy: line 1327: error: assumption failed	1
	bind failed: Address already in use	1
	quic_connection.ivy: line 768: error: assumption failed	249
	quic_connection.ivy: line 732: error: assumption failed	4
	quic_server_test.ivy: line 629: error: assumption failed	177
	quic_connection.ivy: line 1047: error: assumption failed	2
		899/1000
quic_client_test_max	Ran out of values for type cid	862
	quic_client_test.ivy: line 454: error: assumption failed	4
	cipher for level 2 is not set	369
	quic_connection.ivy: line 753: error: assumption failed	21
	quic_connection.ivy: line 1062: error: assumption failed	4
	quic_client_test.ivy: line 517: error: assumption failed	3
	cipher for level 3 is not set	629
	Segmentation fault	105
	timeout: the monitored command dumped core	105
		862/1000

[6] `https://docs.google.com/spreadsheets/d/1WkqKCKSSSM3QD5_SshoD6J30v8t9DslmmHi1xmDJcsU/edit?usp=sharing`

Table 5: **VM Ubuntu - `Quant`**

Test	Error code	#
quic_server_test_stream	quic_connection.ivy: line 1612: error: assumption failed	612
	quic_connection.ivy: line 688: error: assumption failed	1
	bind failed: Address already in use	1
		614/1000
quic_server_test_max	quic_connection.ivy: line 732: error: assumption failed	6
	bind failed: Address already in use	1
	quic_server_test.ivy: line 518: error: assumption failed	128
	quic_server_test.ivy: line 629: error: assumption failed	45
	quic_connection.ivy: line 1612: error: assumption failed	523
		703/1000
quic_server_test_connection_close	quic_connection.ivy: line 1047: error: assumption failed	1
	quic_connection.ivy: line 1327: error: assumption failed	3
	quic_connection.ivy: line 1612: error: assumption failed	106
	bind failed: Address already in use	1
	quic_connection.ivy: line 768: error: assumption failed	342
	quic_connection.ivy: line 732: error: assumption failed	3
	quic_server_test.ivy: line 629: error: assumption failed	310
	quic_connection.ivy: line 1414: error: assumption failed	149
		915/1000
quic_client_test_max	timeout	**1000/1000**

Table 6: **VM Ubuntu - `MinQUIC`**

Test	Error code	#
quic_server_test_stream	timeout	100%
quic_server_test_max	timeout	100%
quic_server_test_connection_close	timeout	100%
quic_client_test_max	timeout	100%

Timeout after receiving "`< cipher_packet()`" event

2.2.2 Explanation of errors

Finally, here are some explanation about the error codes that we got when we launch the test for draft 18:

Path challenge frames (QUIC18-19.17)

- `quic_connection.ivy: line 1612`

  It seems that this error happened when "Path Challenge" Frames which are used to request verification of ownership of an endpoint by a peer is required but failed and is never received in reality. This frame is used to check reachability to the peer and for path validation during connection migration but also in order to minimize the risk of amplification attacks.

Application Close frames (QUIC16-19.4)

- `quic_connection.ivy: line 1414`

This error appears during handling of an "Application close" Frames, it normally requires no (~) initial encryption level but it seems to not be the case here and there is some encryption.

Not strange since not in the current QUIC 18 version.

Max Stream ID frames (QUIC16-19.7)

- `quic_connection.ivy: line 1327`

  This happen during the handling of "Max Stream ID" Frames, when the requirements that these frames may not occur in initial or handshake packets is not respected.

  Not strange since not in the current QUIC 18 version.

RST Stream frames (QUIC18-19.4)

- `quic_connection.ivy: line 1257`

  This happen during the handling of "RST Stream" Frames, when the requirements that the stream id must not exceed the maximum stream id for stream kind is not satisfied.

Stream frames (QUIC18-19.8)

- `quic_connection.ivy: line 1069`

  This happen during the handling of "Stream" Frames that carry stream data, when the requirements that indicate that if the stream length (a) for a destination's frame is less than the total frame length (b) (offset included), then we should update the connection total data with the difference between (a) and (b) added to that but this is not satisfied.

- `quic_connection.ivy: line 1062`

  In theory, the stream ID must be less than or equal to the max stream ID for the kind of the stream. (QUIC16-19.7 not present anymore in the QUIC 18)

- `quic_connection.ivy: line 1047`

  This happen during the handling of "Stream" Frames that carry stream data, when the requirements that indicate that the connection must not have been closed the the source endpoint during the treatment of stream frame but this is violated here.

Packet Protocols

- `quic_connection.ivy: line 768`

  This may occur during the handling of packet events, we expect that during the draining state, we should make sure that a draining packet has not been previously sent and that the packet contains a "Connection Close" frame but this is not the case here. (QUIC18-10.1)

- `quic_connection.ivy: line 753`

  Also occurs during the handling of packet events, we have the requirement that a packet containing only "ACK" frames and padding is "ACK-only" (QUIC18-13.1.1.). For a given CID, the number of ACK-only packets sent from the source to destination must not be greater than the number of non-ACK-only packets sent from destination to the source which is not respected here.

- `quic_connection.ivy: line 732`

  Here again, we have the requirement during the handling of packet event that for a given connection, a server must only send packet to an address that at one time in the past sent the AS packet with the highest numbered packet thus far received. This concern the connection migration address (QUIC18-9) and path challenge frame (QUIC18-19.17).

  Note of McMillan *On see a packet form a new address with the highest packect number see thus far, the server detects migration of the client. It begins sending packets to this address and initiates path validation for this address. Until path validation succeeds, the server limits data sent to the new address. Currently we cannot specify this limit because we don't know the byte size of packets or the timings of packets. On detecting migration, the server abandons any pending path validation of the old address. We don't specify this because the definition of abandonment is not clear. In practice, we do observe path challenge frames sent to old addresses, perhaps because these were previously queued. QUESTION:* ***abandoning an old path challenge could result in an attacker denying the ability of the client to migrate by replaying packets, spoofing the old address****. The server would alternate between the old (bogus) and new address, and thus never complete the path challenge of the new address.*

- `quic_connection.ivy: line 698`

  We also have the requirement that no payload can be empty during connection, we should have at least one frame inside (QUIC18-12.4) which is not respected all the times.

- `quic_connection.ivy: line 687`

  Also during the handling of packet event but here it concern the encryption level during the handshake. It requires the long header packet type and `"0x20"` type bits which is used as Latency Spin Bit (QUIC18-17.3) in header but this is not respected here. This bit enables latency monitoring from observation points on the network path throughout the duration of a connection.

Server

- `quic_server_test.ivy: line 629`

  At the end of the test, we check data some date was actually received from the server. This error indicate that no data were produce in total for the connection and a corresponding client CID.

- `quic_server_test.ivy: line 518`

  During the generation of "Connection Close" frames by the server for the client, we are here in the case when the server is not in the "generating" frame state and so should return an error code `"NO_ERROR"`, but this is not the case. (QUIC18-19.19)

Client

- `quic_client_test.ivy: line 517`

  Same than `quic_server_test.ivy: line 518`

- `quic_client_test.ivy: line 454`

  Before handling stream frame, we need to ensure that the source IP address is equal to the server address and than the destination IP address is the client address which is not the case here.

Errors

- `"Segmentation fault"`

  There is some implementation that still have memory errors in their codes.

- `"Ran out of values for type cid"`

  Indicate that all the possible value for the CID has been used ??

- `"bind failed: Address already in use"`

  This happened when the server suddenly restart or something like that and that the IP address is already used by the last time. This should not happened.

- `"timeout"`

  Timeout launch by Ivy itself, it mean that 100 seconds has been reach and that nothing happened for the QUIC implementation, the tested implementation is either in wrong version, or the tested features is not implemented. Concerning `MinQUIC`, we think that the timeout is caused by wrong TLS implementation since it blocks at the cypher event.

- `"cipher for level X is not set"`

  Error with the TLS encryption, only occurs with `Quant`.

2.3 Comparison between our results and McMillan results

As said before, we perform our test in a similar fashion than McMillan, i.e that we run these test in batches of 1000 on the same server instance to test their long-run behavior too but on 3 machines to see any potential difference.

In a first overlook, it seems that we have more or less the same result than McMillan. We got in total **21** errors, all types included where McMillan revealed 27 errors so either we do not find all of them, either there are some minor difference between our version that correct some of them and also due to the fact that the `MinQUIC` implementation doesn't work for us, and finally since we only test draft 18.

We can find the complete details of McMillan results here.[7] Note that most of the tested version were anterior to the draft 18.

McMillan found that many protocols errors were due to unimplemented requirement. We got more or less the same result, we see that:

- One implementation, `PicoQUIC`, failed to respect a prohibition on sending a packet solely to acknowledges an ACK-only packet.
- Also two of them (`PicoQUIC` and `Quant`) failed to obey a rule that allows client migration to be detected only if a packet with the highest-observed sequence number arrives from a new address.
- We also have one implementation, `Quant`, that did not respect the error code in `CONNECTION_CLOSE` frame.

The other error seems to be more or less the same but to be honest we did not have the time for now to analyse all the resulting `.out` and `.err` files produced. We note that we should automatize that for the future. We also have some trouble for now to classify all the problems detected between protocols errors, vulnerabilities (it seems that we also have the vulnerability of the `PATH_CHALLENGE` that may permit DDoS

[7] `https://github.com/microsoft/ivy/blob/quic18_client/doc/examples/quic/anomalies`

attack) and ambiguity's problems in the specification and crashes (segfault and some timeout).

Also note that some test might complete without signaling error but still some strange behavior are possible. However we did not have time and knowledge for now to detect that in all the produced files.

3 Conclusion

This part of the work allow us to have an overview of Ivy and see that it was a viable solution to perform formal specification testing. We think that for extension with good and clear specification, we could test them in a similar fashion with `Ivy`.

We also concluded that the creation of rules for each version of QUIC was relatively headache especially if you have to redo a good part of the tests every 6 months which is why we plan to automate everything a little bit, but we still need to think about how to do this.

We seems to have similar result than McMillan, but due to our low skill and knowledge for know of all the requirement for the protocol in many of its version, it is difficult for us to analyse efficiently all the errors produces but that show us where we will focus for the next time.

We also learn that the QUIC documentation was indeed very long and not always clear, identifying the testable part, in a formal way, of the draft 28, as McMillan did for the draft 18 and older version, is for the next of the work since now we use all made test of McMillan.

3.1 Next in the work

What we will do now is to:

1. Extend Ivy to be functional with draft 28 of QUIC and make the same test but this time with more QUIC implementation like the chromium one. This will allow us to really go deep into the specification of the current version of QUIC.
2. Extend Ivy to be functional with QUIC extensions and see what is possible
3. To determine, either make Ivy more usable for future version of QUIC, either exploring other possibility such as eBPF. To be determined and discussed.

Or what we could also do is to continue on this QUIC version and directly see if some QUIC extensions for this version can be tested with `Ivy`. This option could be chosen since in a first view adapting Ivy for current draft will take time and we don't know if this is really useful for the purpose of testing QUIC extensions.